
Chapitre n° 10

IMAGES

1 PIXEL MON AMI

Une image de type *bitmap* (tableaux de bit) est un tableau de *pixels* (PICture ELements).

Un pixel contient de l'information sur la couleur qu'il représente :

- Une nuance de gris de 0 à 255, ou entre 0 et 1, ou un nombre hexadécimal de 00 à FF
- Trois niveaux de 0 à 255 (ou 0 à 1, ou en hexadécimal) en RBG : Red/Green/Blue
- Éventuellement 4 niveaux en RGBA : Alpha pour la transparence

Le poids d'un fichier Bitmap est :

$1 \times L \times H$ octets en Niveau de gris

$3 \times L \times H$ octets en RGB

$4 \times L \times H$ octets en RGBA

La numérisation d'image est simple : on découpe ce que l'on voit en 'petits morceaux' et on détermine au mieux la couleur du pixel. Plus on découpe finement, plus l'image est proche du réel, mais plus le poids est grand.

Le Poids d'une image Bitmap peut être assez lourd, pour cela on a recourt à de la compression de données, avec ou sans perte d'information (destructive ou non).

- Le .bmp est le format brut : pas de perte, pas compressé
- Le .png est compressé, mais non destructif
- Le .gif pareil avec une autre méthode
- Le .jpg est compressé et destructif

L'autre façon de définir une image est par des courbes qui représentent les formes qui la compose.

Avantage : Facile de zoomer, contrairement au bitmap. Désavantage : C'est bien plus difficile à coder!

Nous ne travaillerons qu'avec du Bitmap.

2 POIDS FORT ET FAIBLE

Pour un nombre en hexadécimal à 2 chiffres $n = \overline{XY}$ avec $X, Y \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ on appelle X le chiffre de **poids fort** et Y le chiffre de **poids faible**. C'est lié au fait que $n = X \times 16 + Y$ en décimal.

Notons aussi que si n est écrit en décimal : $X = n // 16$ et $Y = n \% 16$.

Pourquoi cette notion est importante? si n est le niveau d'une couleur la différence entre $\overline{XY}$ et $\overline{X0}$ est minime, ce qui peut permettre :

- De réduire le poids d'une image : on supprime le chiffre de poids faible.
- De "cacher" de l'information, en plaçant celle à la place du chiffre de poids faible.

Un nombre hexadécimal de deux chiffres c'est un nombre décimal entre 0 et 255, c'est aussi un nombre binaire à 16 chiffres. On peut aussi adapter cette idée en utilisant le bit de l'unité (0 ou 1), appelé **bit de poids faible**. La différence sur une couleur est encore plus minime. En faisant cela sur chaque pixel, il devient très facile de dissimuler un information écrit en binaire dans une image.

3 TRAITEMENT

En Python, il existe de nombreuses bibliothèques pour travailler avec les images. Nous irons au plus simple : ce bon vieux matplotlib avec `matplotlib.image` et l'aide `numpy`. Mais il existe aussi la bibliothèque `Pillow`. Les positions des pixels commencent en haut à gauche.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.image as mpimg
4
5 >>> mpimg.imread("vacances.png")
6 >>>
7 array([[0.07058824, 0.          , 0.14901961],
8        [0.11372549, 0.03529412, 0.22745098],
9        ...,
10       [0.48627451, 0.45882353, 0.42745098]] , dtype=float32)
```

Avec cette manipulation les images sont des `array` de `numpy`, cela fonctionne comme des tableaux de tableaux pour de dont on a besoin.

Un traitement **local** est un traitement pixel par pixel.

On applique une fonction $f : [0, 1] \rightarrow [0, 1]$ à chaque pixel.

- Modification du contraste
- Éclaircissement ou assombrissement
- Passage en niveau de gris ou noir et blanc
- Des trucs marrants

Comment parcourir un tableau pixel par pixel par couleur : 3 boucles `for` ! C'est pour ça que le traitement d'images peut être un peu long :

```
1 for i in range(hauteur):
2     for j in range(largeur):
3         for k in range(3):
```

Il suffit de parcourir un tableau de tableau et d'appliquer f à chaque pixel ou chaque composante.

Un traitement **global** est un traitement sur la matrice des pixels, chaque résultats dépends de la position et du/des voisins.

La fonction $f : L \times H \rightarrow L \times H$ doit être bien choisie...

- Floutage
- Zoom ou dilatation
- Des trucs marrants

Il suffit de parcourir un tableau de tableau et d'appliquer f selon les coordonnées ou les voisins, c'est une question plus compliquée car plus rigoureuse